

Chapter 17 and Chapter 25

GRASP: Designing Objects with Responsibilities



GRASP

- General Responsibility Assignment Software Patterns (GRASP)
 - Craig Larman's 9 principles for software design => 설계 원칙
- Typical tasks in constructing a software with OOAD
 - Identify the requirements
 - Create a domain model
 - Create design model
 - Add attributes and methods to the software classes
 - Define the interactions between the objects
 - Okay, I got it. But, how ???
- GRASP
 - will guide you how to assign responsibilities to collaborating objects



Responsibilities

- Responsibility = "a contract or obligation of a class"

① **Knowing** responsibilities:

- knowing about private encapsulated data
- knowing about related objects
- knowing about things it can derive or calculate
- Ex) "a *Sale* is responsible for *knowing its total*"

② **Doing** responsibilities:

- doing something itself
- initiating action in other objects
- controlling and coordinating activities in other objects
- Ex) "a *Sale* is responsible for *creating SalesLineItems*"

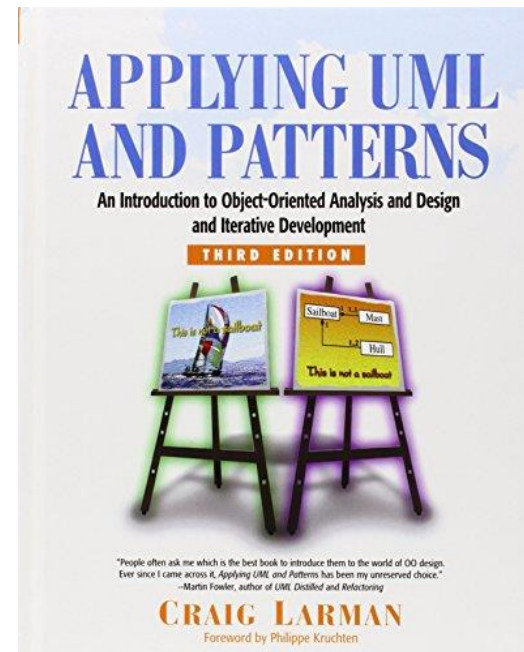
- A responsibility is not the same as a method, but **methods are implemented to fulfil responsibilities.**

responsibility ≠ method. (1:1 mapping은 아님).



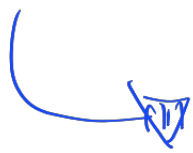
GRASP: General Principles in Assigning Responsibilities

- From Craig Larman's 9 principles:
 - Information Expert
 - Creator
 - Controller
 - Low Coupling
 - High Cohesion
 - Polymorphism
 - Pure Fabrication
 - Indirection
 - Protected variations



Modularity

- Goal of design *여러 module로 분할.*
 - partition the system into modules and assign responsibility among the components
- Modularity reduces the total complexity a programmer has to deal with at any one time:
 - functions are assigned to modules in a way that groups similar functions together (**Separation of Concerns**), and
 - there are small, simple, well-defined interfaces between modules (**Information Hiding**)
- Measuring Modularity
 - Coupling: measure of the inter-dependencies between modules
 - Cohesion: measure of how strongly the elements in a module are related

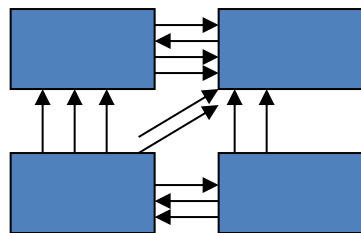
 coupling ↓ ,
Cohesion ↑ *조금.*



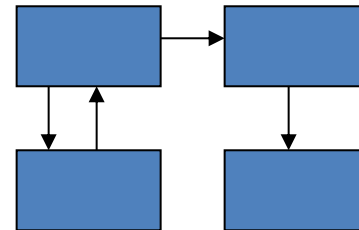
Concept of Coupling

■ Coupling

- measure of the inter-dependencies between classes (or modules)
- a class with high (or strong) coupling relies on many other classes.
- when the independent element changes, it affects the dependent one.



Highly coupled



Loosely coupled

Pursue Low (or loose) Coupling!

• Disadvantages of High Coupling:

- Forced changes due to changes in depending classes
- Harder to understand in isolation
- Harder to reuse

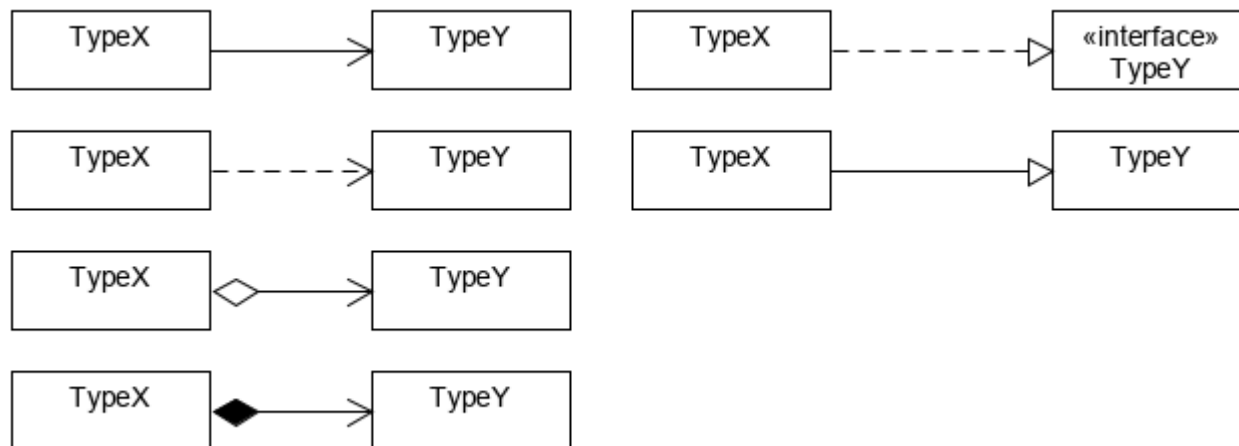
↳ loosely coupling $\frac{1}{2}$ (1 1 1 2)



Common forms of coupling in OO languages

TypeX depends on TypeY if

- TypeX has an **attribute referencing** a TypeY instance, or TypeY itself.
- TypeX has a **method referencing** a TypeY instance, or TypeY itself, by any means. (e.g. a parameter or local variable of type TypeY, or the returned object is an instance of TypeY, etc)
- TypeX is a direct or indirect **subclass** of TypeY.
- TypeX **implements** interface TypeY.

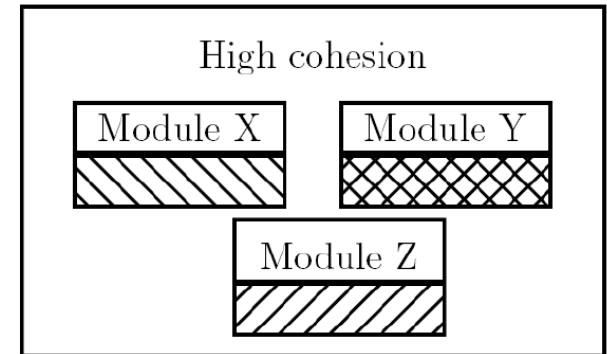
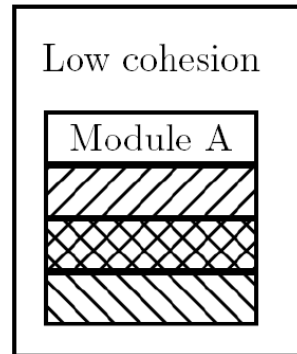


Concept of Cohesion

■ Cohesion

- measure of how strongly the elements in a class (module) are related
- the **related responsibilities** are placed into one manageable unit.
- related to Single Responsibility Principle (SRP)

하나의 responsibility가 하나의 method에 대응될때 best



Pursue High Cohesion!

- Advantages of High Cohesion = coupling ↓ (trade-off)
 - Easily understandable and maintainable
 - Code reuse
 - Low coupling

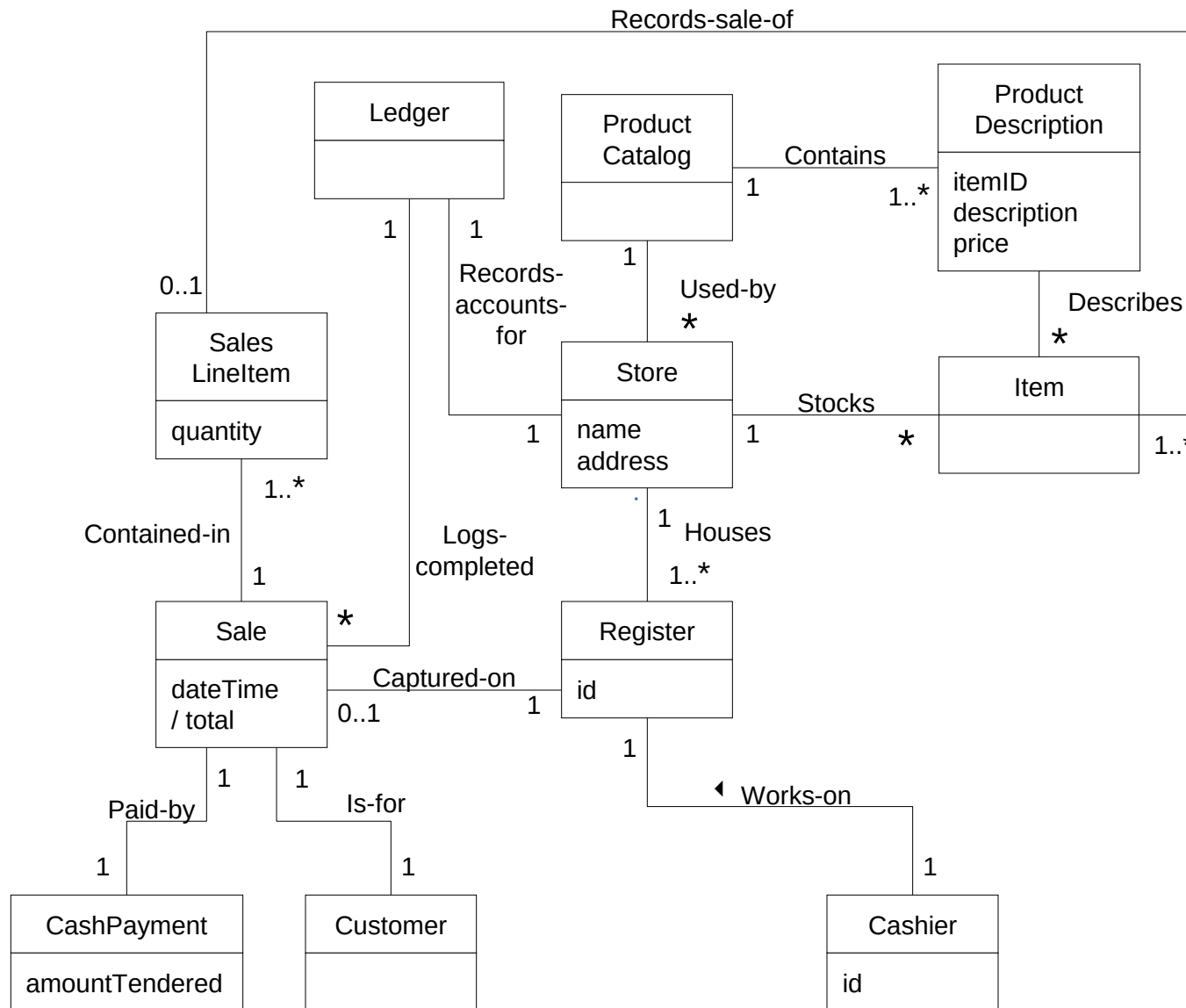


Case Study – Point-Of-Sale (POS) System

- is used to record sales and handle payments; it is typically used in a retail store.
- includes hardware components such as a computer and bar code scanner, and software to run the system.
- interfaces to various service applications, such as a third-party tax calculator and inventory control.



Domain Model



Creator Pattern

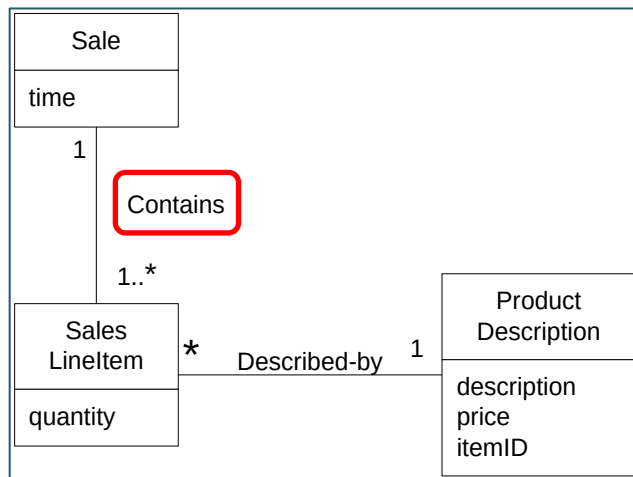
Name	Creator
Problem	Who creates an A?
Solution	Assign class B the responsibility to create an instance of class A, if one of these is true (the more the better): • B contains or aggregates A. • B records A. • B closely uses A. • B has the initializing data for A. if there is a choice, prefer “B contains or aggregates A”

Creator Example

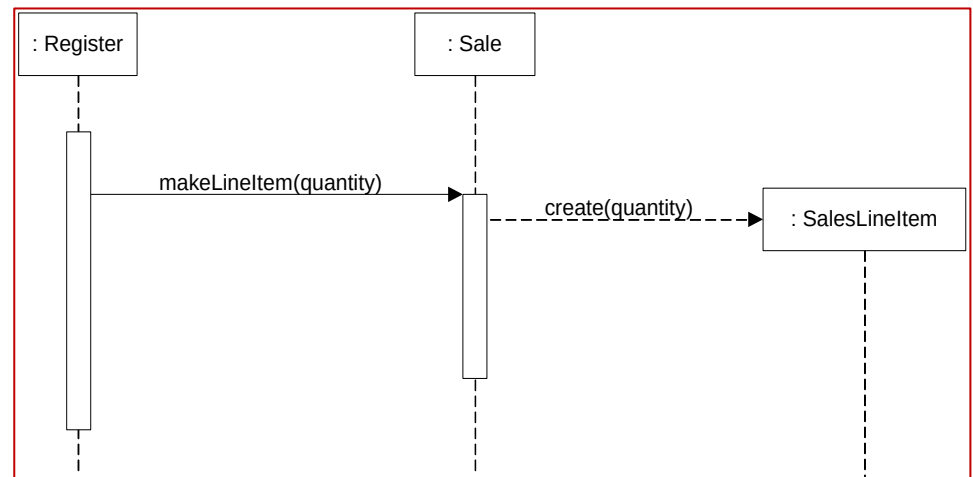
- Who should be responsible for creating a *SalesLineItem* instance?

Assign class B the responsibility to create an instance of class A, if :

- B contains or aggregates A.
- B records A.
- B closely uses A.
- B has the initializing data for A.



Domain Model



Design Model

About Creator Pattern

- **Discussion:**

- Guides assigning creation responsibilities
- Contradictions: creation may require significant complexity
 - using recycled instances for performance reasons
 - conditionally creating an instance from one of a family of similar classes
 - sometimes desired to outsource object wiring (“dependency injection”)

- **Benefits:**

- Low Coupling is supported

- **Related patterns:**

- Abstract Factory, Singleton



Information Expert Pattern

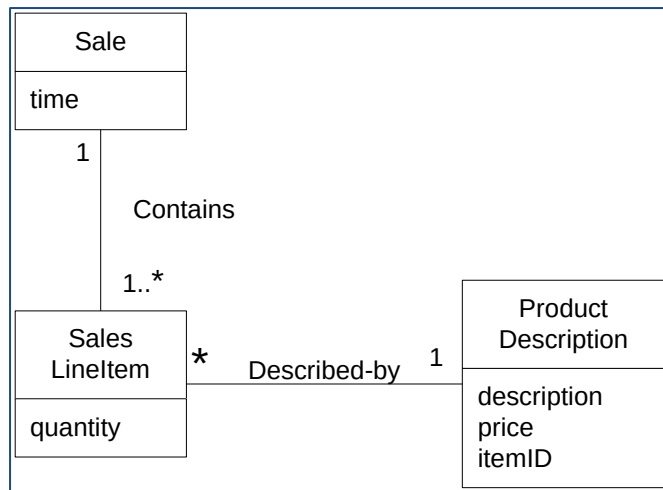
일반적인 상황.

Name	Information Expert
Problem	What is a basic principle by which to assign responsibilities to objects?
Solution	Assign a responsibility to the class that has the information needed to fulfill it.

Information Expert Example

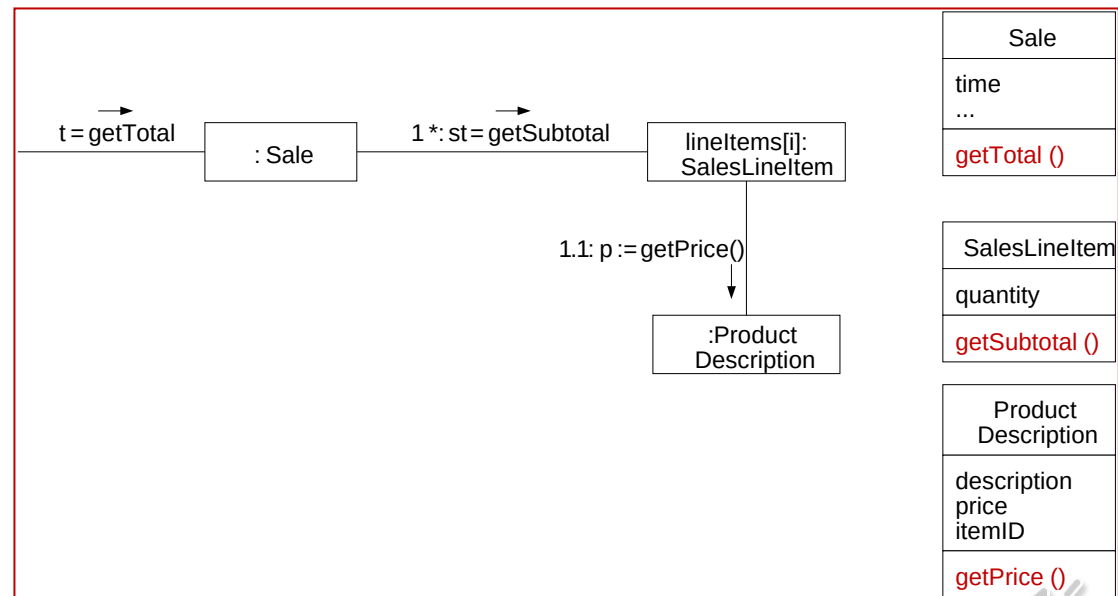
- Who should be responsible for knowing the grand total of a sale?

Assign a responsibility to the class that has the information needed to fulfill it.



Domain Model

Design Class	Responsibility
Sale	knows sale total
SalesLineItem	knows line item subtotal
ProductDescription	knows product price



Design Model

About Information Expert Pattern

▪ Discussion

- **Most used principle** in the assignment of responsibilities
- Information is spread across different objects -> they need to interact
- Caution: May conflict with Separation of Concerns
 - Example: Responsibility for saving a sale in the database
 - Adding this responsibility to Sale
 - distribute database logic over many classes : low cohesion

▪ Benefits

- Information encapsulation → supports low coupling



Controller Pattern

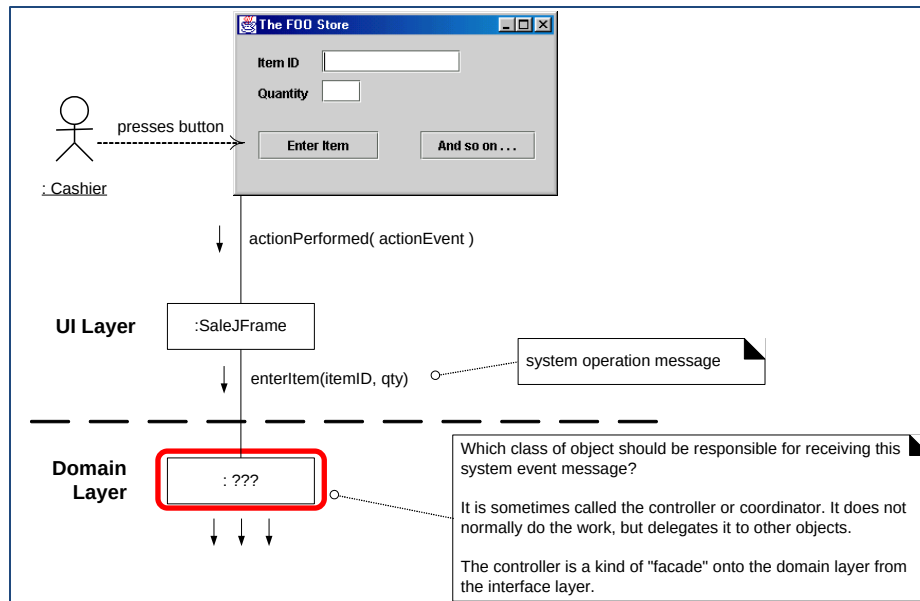
Name	Controller
Problem	What first object beyond the UI layer receives and coordinates ("controls") a system operation?
Solution	Assign the responsibility to an object representing one of these choices: • 1st option: Overall "system," a "root object," a device that the software is running within, or a major subsystem (<i>facade controller</i>) • 2nd option: Use case scenario within which the system operation occurs (<i>use case</i> or <i>session controller</i>)

Controller Example

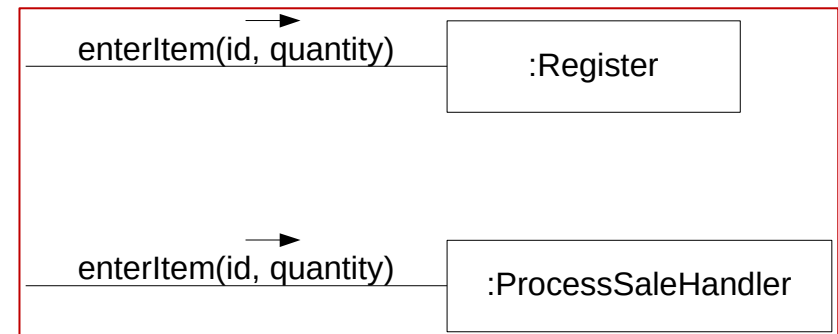
- Who should be the controller for system events such as `enterItem` and `endSale`?

Assign the responsibility to an object representing one of these choices:

- 1st option: Overall "system," a "root object," a device that the software is running within, or a major subsystem (these are all variations of a *facade controller*).
- 2nd option: Use case scenario within which the system operation occurs (a use case or *session controller*)



By the Controller pattern, two choices:



Design Model

Controller Example

▪ Choices

- Register (or POSSystem): represents the overall "system", device, or subsystem
→ Façade controller
- ProcessSaleHandler (or ProcessSaleSession): represents a receiver or handler of all system events of a use case scenario or session
→ Use-case controller (or session controller)



About Controller

▪ Discussion

- Controller class is called bloated if overloaded with too many responsibilities
 - Add more controllers (façade -> use-case controllers)
 - Delegates to other objects

▪ Benefits

- No application logic in the GUI
- Increased potential for reusable components
- Reason about the state of the use case
 - Useful when operations must be performed in a specific order
Example) MakePayment cannot occur until EndSale has occurred



Low Coupling Pattern

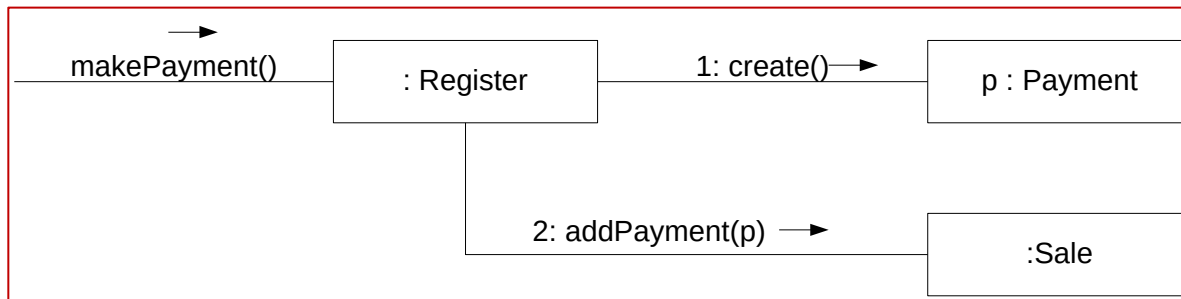
Name	Low Coupling
Problem	How to reduce the impact of change?
Solution	Assign responsibilities so that (unnecessary) coupling remains low. Use this principle to evaluate alternatives .

Low Coupling Example

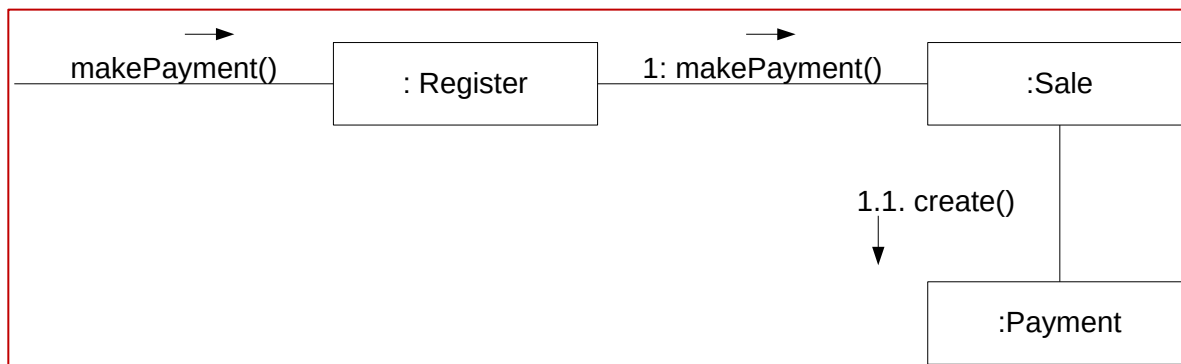
- We need to **create a Payment instance** and **associate it with the Sale**. What class should be responsible for this?

Assign responsibilities so that (unnecessary) coupling remains low
Use this principle to **evaluate alternatives**.

- Option 1: Register "records" a Payment in the real-world domain, the **Creator pattern suggests Register** as a candidate for creating the Payment



- Option 2: Sale creates the *Payment* and associate it with the *Sale*



About Low Coupling Pattern

▪ Discussion

- Low coupling encourages designs to be more independent which reduces the impact of change.
- Needs to be considered with other patterns such as Information Expert and High Cohesion.
- High coupling to stable “global” objects is not problematic
Example) coupling to Java libraries such as `java.util`



High Cohesion Pattern

Name	High Cohesion
Problem	How to keep objects focused, understandable, and manageable, and as a side effect, support Low Coupling?
Solution	Assign responsibilities so that cohesion remains high. Use this to evaluate alternatives.

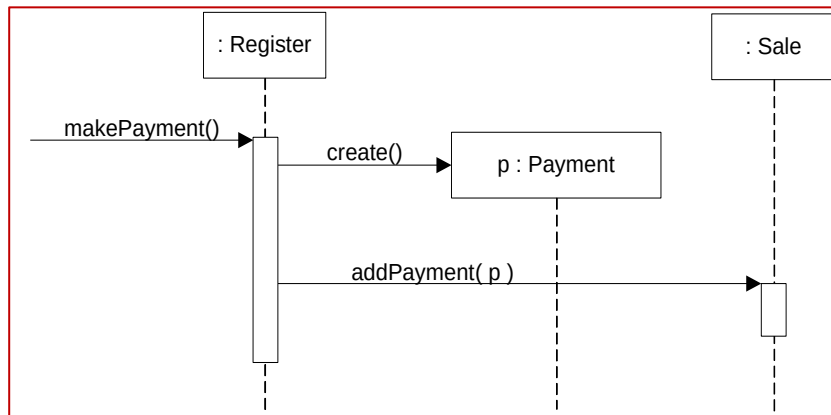
High Cohesion Example

- Where to place the responsibility for making a payment?

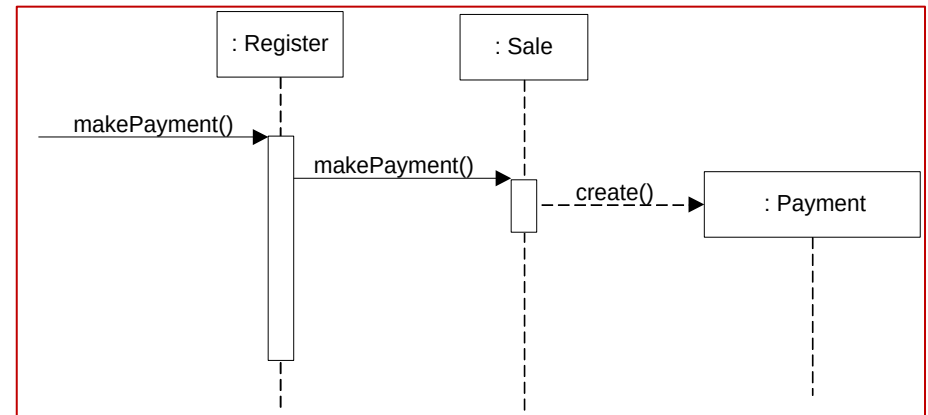
Assign responsibilities so that cohesion remains high.
Use this principle to **evaluate alternatives**.

Option 1: Assign to the Register. In this isolated example, this is acceptable; but the Register will get bigger

Option 2: Delegates the payment creation responsibility to the Sale



Option 1



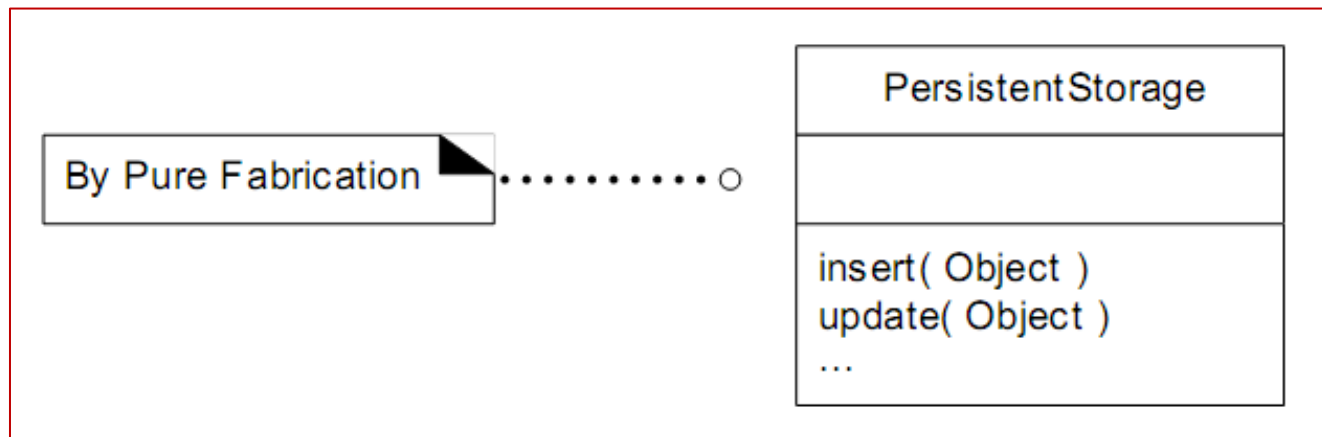
Option 2 😊

Pure Fabrication Pattern

Name	Pure Fabrication
Problem	Who is responsible when you are desperate, and do not want to violate high cohesion and low coupling?
Solution	Assign a highly cohesive set of responsibilities to an artificial or convenience "behavior" class that does not represent a problem domain concept something made up , in order to support high cohesion, low coupling, and reuse .

Pure Fabrication Example

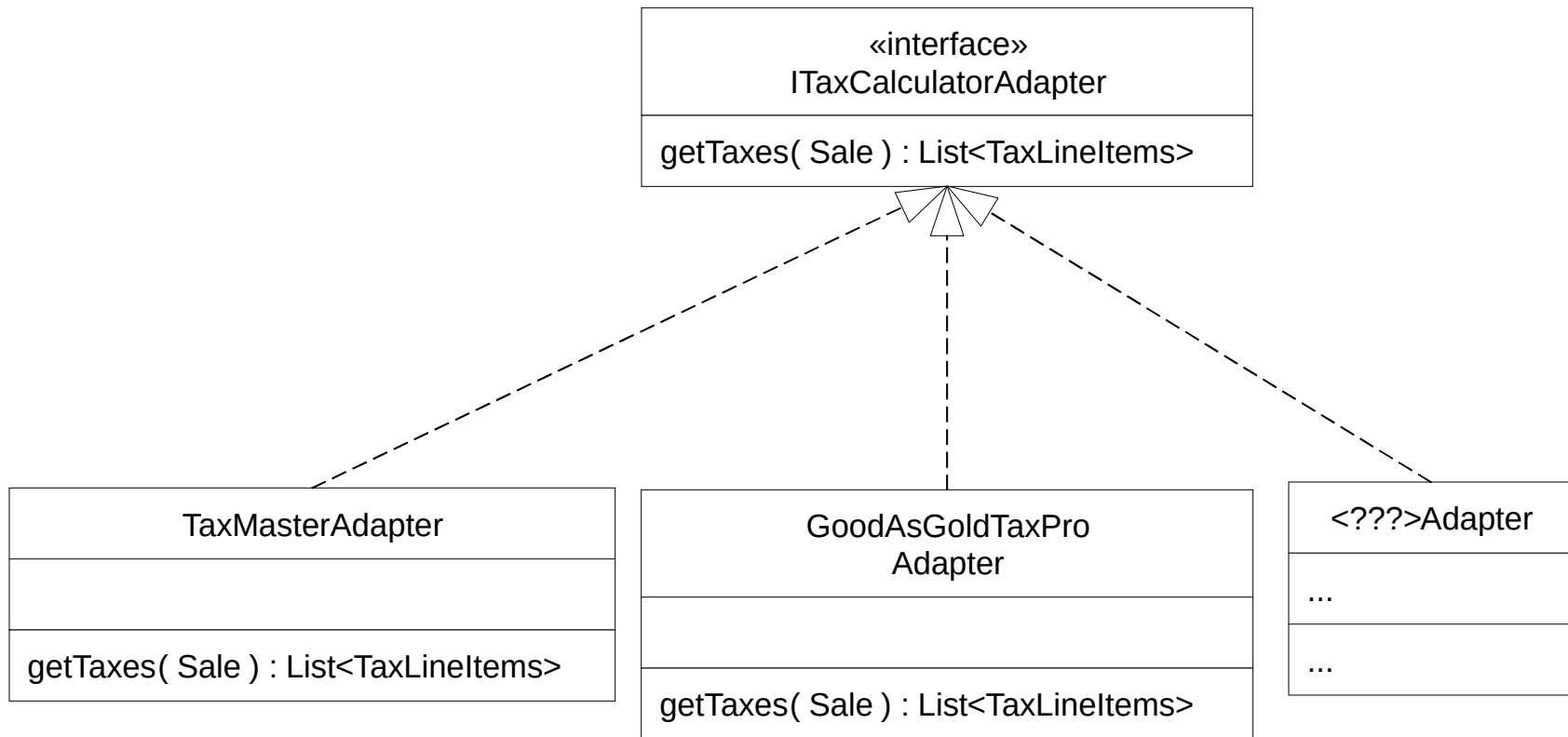
- We want to save Sale instances in a DB.
 - By Information Expert, let's assign this responsibility to Sale class.
 - the task requires many DB related operations and the Sale class becomes **low cohesive**.
 - **poor reuse** or **lots of duplication** in other classes that do the same thing.
 - By Pure Fabrication
 - **create a new class** that is **solely responsible for saving** objects in a persistent storage medium



Polymorphism Pattern

Name	Polymorphism
Problem	Who is responsible when behavior varies by type?
Solution	When related alternatives or behaviors vary by type (class), assign responsibility for the behavior using polymorphic operations to the types for which the behavior varies.

Polymorphism Example



By Polymorphism, multiple tax calculator adapters have their own similar, but varying behavior for adapting to different external tax calculators

About Polymorphism Pattern

- Benefits
 - Easier and more reliable than using explicit selection logic
 - Easier to add additional behaviors later on

- Costs
 - Increases the number classes in a design
 - May make the code less easy to follow

- Discussion
 - fundamental principle in design: how to handle similar variations.
 - Avoid excessive use for “future-proofing” against yet unknown potential future variations
 - Agile approach: no significant “upfront design” and add the variation point only when the need arises

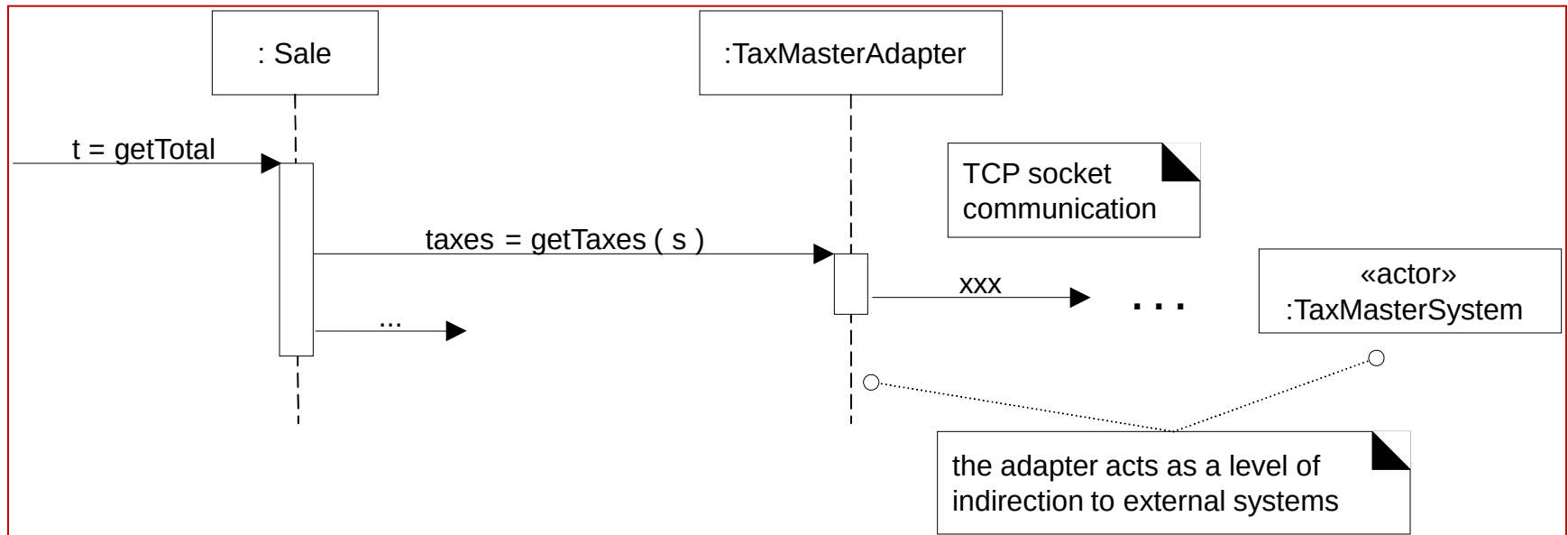


Indirection Pattern

Name	Indirection
Problem	How to assign responsibilities to avoid direct coupling?
Solution	Assign the responsibility to an intermediate object to mediate between other components or services, so that they are not directly coupled.

Indirection Example

- By adding a level of **indirection** and adding **polymorphism**, the adapter objects protect the inner design against variations in the external interfaces



About Indirection Pattern

- How can we avoid a direct coupling between two or more elements?
- Indirection **introduces an intermediate units** (classes) to communicate between the other units, so that the other units are not directly coupled.
- Benefits
 - Low Coupling
- Example: Adapter, Facade, Proxy, Mediator, ...



Protected Variations Pattern

Name	Protected Variations
Problem	How to assign responsibilities to objects, subsystems, and systems so that the variations or instability in these elements do not have an undesirable impact on other elements?
Solution	Identify points of predicted variation or instability Assign responsibilities to create a stable "interface" around them.

About Protected Variations Pattern

- OCP and PV are two expressions of the same principle
 - protection against change to the existing code and design at variation and evolution points with minor differences in emphasis
- Benefits
 - Flexibility and Protection from Variations
 - Provides more structured design
- Mechanisms motivated by PV
 - Uniform access in C#, Ada, and Eiffel
 - Express both method and field access in the same way
 - Service lookup
 - JNDI
 - Reflective or meta-model designs

Summary

- General Responsibility Assignment Software Patterns (GRASP)

- Information Expert
- Creator
- Controller
- Low Coupling
- High Cohesion
- Polymorphism
- Pure Fabrication
- Indirection
- Protected variations

